



Tribhuvan University
Faculty of Humanities and Social Science
Swastik college, Bhaktapur



Distributed System
Case Study on
Java RMI and Message Passing Interface (MPI)

Submitted By:
Safalta Shrestha (113102089)
BCA 6th Semester

Submitted To:
Sagar Rana Magar
Lecturer, BCA Department
Swastik college, Bhaktapur
January 2025

Table of Content

Executive Summary.....	III
Chapter-1 Introduction and Background.....	1
1.1 Introduction.....	1
1.2 History and Timeline.....	2
1.3 Key Features of Java RMI & MPI.....	3
1.4 Parameter Passing in RMI.....	3
1.5 Communication Model of MPI.....	3
1.6 Objective.....	4
Chapter-2 Architecture / Layers of Java RMI & MPI.....	5
Chapter-3 Working of Java RMI & MPI	8
Chapter-4 Applications / Use Cases.....	11
Chapter-5 Advantage and Limitation.....	14
Chapter-6 Real-Time Case Examples.....	16
Chapter-7 Conclusion.....	17
References.....	18

Executive Summary

Distributed computing has become a fundamental component of modern computer systems, enabling multiple machines to collaborate in solving complex problems efficiently. Java Remote Method Invocation (RMI) and Message Passing Interface (MPI) are important technologies used in distributed computing to enable communication between processes running on different systems. Java RMI provides an object-oriented approach that allows remote method calls between Java applications, making distributed programming easier. On the other hand, MPI uses explicit message passing and is designed for high-performance and parallel computing environments.

This study briefly examines the architecture, communication models, advantages, limitations, and applications of Java RMI and MPI, highlighting their role in modern distributed systems and their suitability for different computing needs

Chapter-1 Introduction and Background

1.1 Introduction

RMI(Remote Method Invocation) is a mechanism that allows an object residing in one Java Virtual Machine(JVM) to invoke methods of an object running in another JVM, possibly on a different machine, over a network. It enables Java objects to communicate remotely, share workload across multiple systems, and invoke methods over a network while automatically handling networking tasks like sockets communication and object serialization.

Java RMI is widely used to develop distributed systems, where different components of a program run on separate machines. For example, in a university system, admission, examination, and library modules may operate on different servers but communicate with each other using Java RMI to share data and services.

Java RMI also fits naturally into client–server applications, where the client requests services and the server provides them remotely. The client invokes methods on server objects as if they were local, while the actual processing takes place on the server side. For instance, in a banking system, the client requests the account balance, and the server processes the request and returns the result to the client through RMI.

The Message Passing Interface (MPI) is a standard for distributed memory parallel computing in which multiple processors communicate with each other by passing messages. It enables efficient communication and coordination among processes running on different computing nodes in a cluster or distributed system. MPI provides a set of functions and routines for communication and synchronization, making it easier to develop scalable parallel applications.

Distributed memory parallel computing is a computing model where each processor or node has its own local memory and processors exchange data through a network. This approach is widely used in high-performance computing (HPC) applications such as scientific simulations, weather forecasting, data analysis, and machine learning. MPI is the most commonly used standard for implementing distributed memory parallel computing due to its portability, efficiency, and performance.

1.2 History and Timeline

Java RMI (Remote Method Invocation) was introduced by Sun Micro-systems as part of the Java platform to address the growing need for distributed computing in the mid-1990s. Before RMI, building distributed applications in Java required developers to manually manage network communication, sockets, and data serialization, which was complex, error-prone, and time-consuming. RMI provided a high-level, object-oriented approach to allow Java objects on different JVMs, possibly on different machines, to communicate seamlessly as if they were local objects.

Year	Event
1996	Java RMI introduced in JDK 1.1
Late 1990s	RMI become popular for distributed client-server applications
2002	RMI-IIOP introduced for CORBA interoperability
2004-2010	Improvements in security, performance and serialization
2010-Present	RMI continues in enterprise systems alongside web-based services

Table 1 Evolution of RMI

Message Passing Interface(MPI) was developed in the early 1990s as a way to enable parallel computing on distributed systems, such as clusters and supercomputers. It provides a set of functions and routines for communication and synchronization between processes and is designed to be both portable and efficient.

Version	Approved	Key Milestones & Features
MPI-1.0	May 1994	The first official standard. Focused on point-to-point and collective communication, and data types.
MPI-1.1	June 1995	Minor updates and bug fixes.
MPI-2.0	July 1997	Major Update: Added Parallel I/O, One-sided communication (Remote Memory Access), and Dynamic Process Management.
MPI-2.1	June 2008	Combined MPI-1.2 and MPI-2.0 into a single document with minor clarifications.
MPI-3.0	Sept 2012	Major Update: Introduced Non-blocking Collectives, enhanced One-sided communication, and Fortran 2008 bindings.
MPI-3.1	June 2015	Minor extensions and significant clarifications.
MPI-4.0	June 2021	Major Update: Introduced Sessions (an alternative way to initialize MPI), Partitioned Communication, and "Large Count" support for data exceeding 2^{31} elements.
MPI-4.1	Nov 2023	Refined Persistent Collectives and improved error handling definitions.
MPI-5.0	June 2025	Approved recently; introduces a standardized Application Binary Interface (ABI) to make compiled MPI programs portable across different MPI implementations.

Figure 1 Evolution of MPI

1.3 Key Features of Java RMI & MPI

- Object-Oriented Remote Communication: Allows Java objects to invoke methods on remote objects in a transparent way.
 - Automatic Serialization: RMI automatically serializes and deserializes objects for transmission over the network.
 - Simplified Networking: Developers do not need to manage sockets, protocols, or data marshaling manually.
 - Exception Handling: Remote method calls can throw exceptions, which are automatically propagated to the client.
 - Platform Independence: RMI works across different operating systems supporting JVMs.
-
- Message-based communication between processes
 - High performance and scalable for large systems
 - Language-independent (C, C++, Fortran, Java)
 - Designed for distributed memory architectures

1.4 Parameter Passing in RMI

Java RMI uses Java's object serialization mechanism for parameter passing. Two types:

1. Pass-by-Value

In Java RMI, Serializable objects are passed by value so changes on the remote side do not affect the original. E.g, a student object

2. Pass-by-Reference

Remote objects are passed by reference via a stub and method call affect the actual object on the server. E.g, a bank account object

1.5 Communication Model of MPI

The Message Passing Interface(MPI) uses a message passing communication model for distributed memory systems. MPI supports two main types of communication: point-to-point and collective communication and uses communicators and ranks to manage process interactions.

A. Point-to-Point Communication

This is the simplest form of communication involving only two processes: a sender and a receiver.

Send Operation: The source process sends a message (data) to a specific destination process ID (rank).

Receive Operation: The destination process must issue a corresponding receive command to "catch" the data into its local buffer.

Blocking vs. Non-blocking: In blocking communication, the process waits until the transfer is complete. In non-blocking, the process continues executing other tasks while the data transfer happens in the background.

B. Collective Communication

Unlike point-to-point, collective communication involves all processes within a defined group (called a Communicator).

Broadcast: One process sends the same data to all other processes.

Scatter: One process splits a data-set and sends different pieces to different processes.

Gather: One process collects pieces of data from all others and combines them.

Reduction: Processes combine their data using a mathematical operation (like Sum, Max, or Min) and store the result in one place.

1.6 Objective

The objective of this case study is to analyze Java RMI & MPI, its architecture, communication model, features, and applications, and to demonstrate how it enables remote communication between Java objects, simplifies client-server development, and supports distributed system design.

Chapter-2 Architecture / Layers of Java RMI & MPI

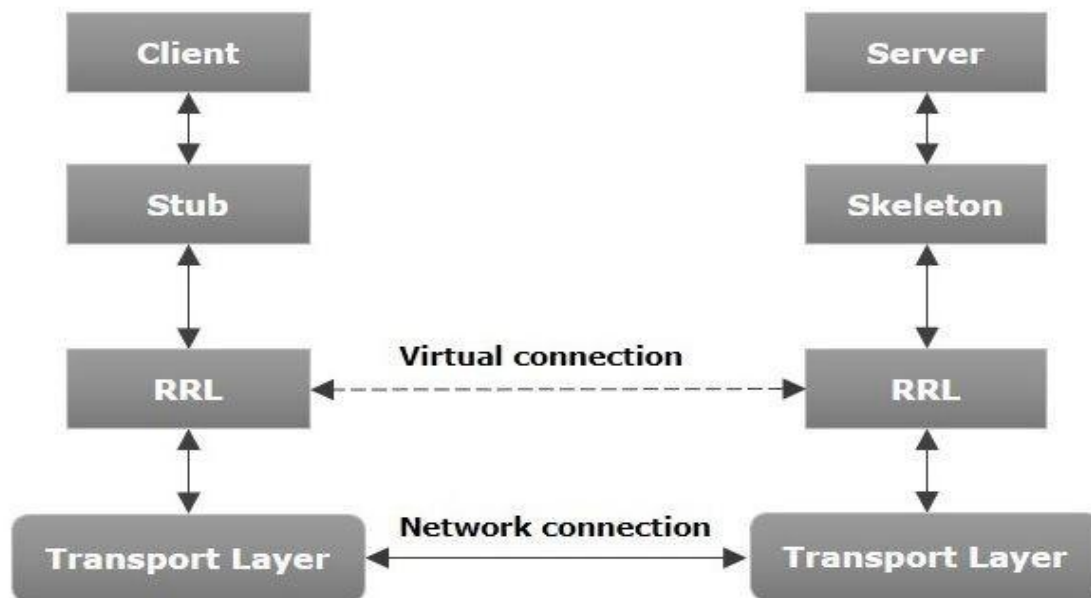


Figure 2 Architecture of RMI

Layers of Java RMI system

- **Stub/Skeleton Layer**

- **Stub:** Client-side proxy object representing a remote object. It facilitates method invocation for clients.

- **Skeleton:** Server-side component receiving method calls from client stubs and forwarding them to the remote object.

- **Remote Reference Layer(RRL):**

- Manages references to remote objects.
 - References act as handles enabling client interaction with remote objects.

- **Transport Layer:**

- Handles network communication between client and server.
 - Encompasses protocols and mechanisms for data transmission.

-Works together with the RMI Registry, which provides a naming service:

1. Allows clients to locate remote objects by name.
2. Servers register remote objects using bind() or rebind() methods.
3. Clients use lookup() to obtain the remote object's stub for communication.

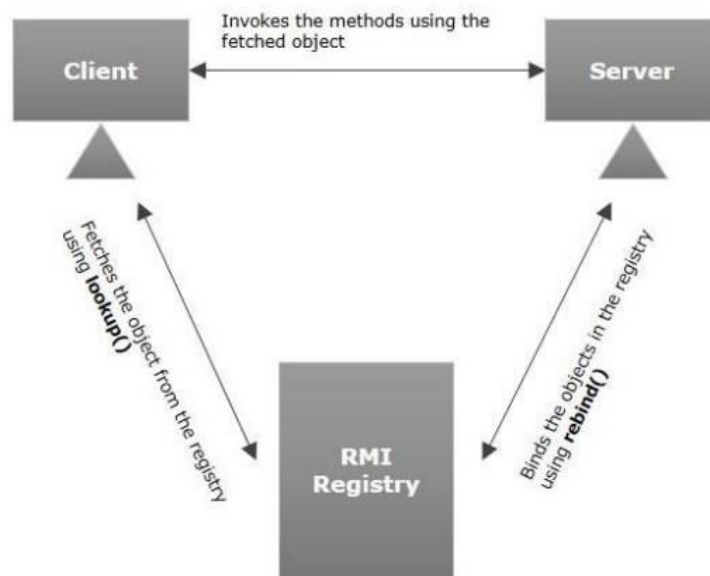


Figure 3 RMI Registry

The diagram(4) shows two processors(nodes) communicating using message passing which is the core idea of MPI.

A. Distributed Memory Architecture

Processor 1 and Processor 2 each have their own local memory. There is no shared memory between the processors. This matches MPI's distributed memory model.

B. Processes

Process A runs on Processor 1 and Process B runs on Processor 2. Each process executes independently and has its own address space. In MPI terms, these processes are identified by ranks.

C. Application Layer

The boxes labeled application send and application receive represent the MPI application layer. Process A sends data using an MPI send operation(e.g,MPI_Send) while Process B receives data using an MPI receive operation(e.g,MPI_Recv).

D. System Buffer

The system buffer represents internal buffering handled by the MPI implementation. Data is temporarily stored here during transmission. This buffering supports efficient and reliable message transfer.

E. Message Passing Communication

The red arrow labeled DATA shows Data send from Process A, passed through the system buffer, transmitted over the network and received into Process B's buffer. This illustrates point-to-point communication.

F. Network /Transport Layer

The connection between the two processors represents the network or interconnect. MPI uses this layer to transfer message using protocols like TCP/IP or high-speed interconnects.

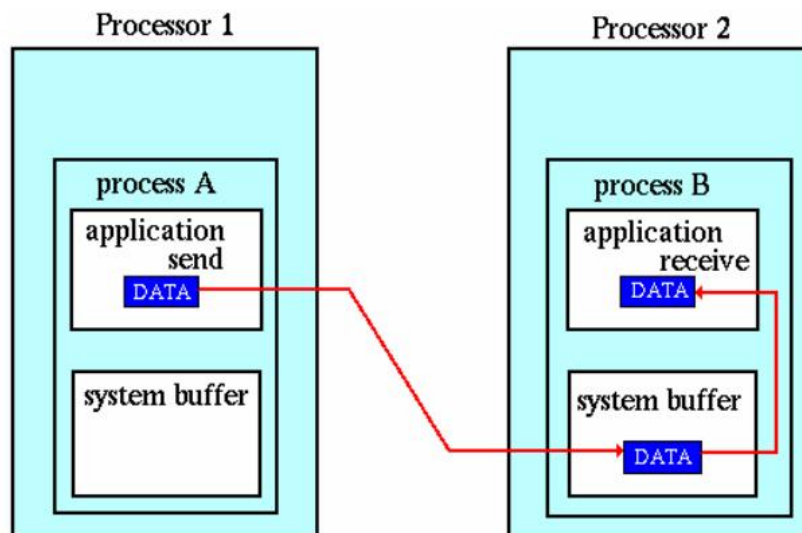


Figure 4 Architecture of MPI

The figure illustrates the architecture of MPI in a distributed memory environment. Each processor executes a separate process with its own local memory. Communication between processes occurs through explicit message passing using send and receive operations. Data is transferred via system buffers and a network interconnect, ensuring no shared memory is required.

Chapter-3 Working of Java RMI & MPI

In RMI, exceptions, object serialization and parameter passing are handled automatically, so developers can focus on application logic. The client-server communication Java RMI can be explained with the help of the figure as follows:

1. Client calls a method on the stub: In the figure, Remote Method Call is shown interacting with the Stub Objects. The stub acts as a client-side proxy for the remote object, receiving the method call from the client application.

2. Stub sends the request through the Remote Reference Layer: The stub packages the method parameters and forwards them to the remote reference layer. In the figure, the stub prepares the parameters and sends them across the network toward the server.

3. Transport layer transmits the request to the server: The Internet icon in the figure represents the transport layer(network), that carries the packaged data to the server-side.

4. Skeleton receives the request and invokes the method: On the server side, the Skeleton objects receive the incoming parameters. The skeleton unpacked the parameters and forward the request to the actual method implementation represented by Remote Method Invocation on Server.

5. Method executes and Result is sent back to the client: After the method executes, the return value is packed by the skeleton and send back over the network. The figure shows arrows labeled return value going back from the server skeleton to the client stub.

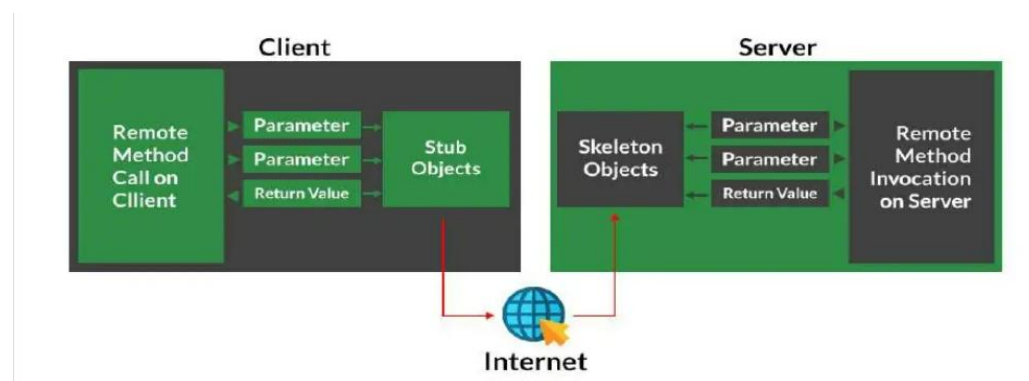


Figure 5 Working of RMI

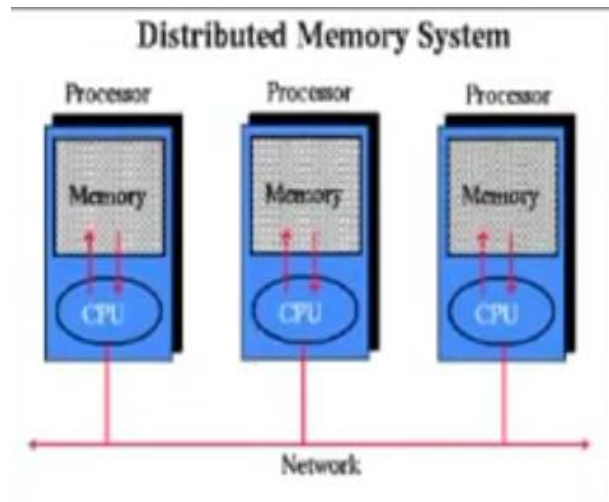


Figure 6 Working of MPI in DS

The given diagram illustrates how MPI operates in a distributed memory system, where multiple processors communicate by exchanging messages over a network.

1. Distributed Memory

In MPI, each processor has its own private memory. There is no shared global memory, so processors cannot directly access each other's data. All data exchange must occur through message passing.

2. Processor

Each processor executes one or more independent processes. These processes run in parallel and perform computations on their local memory.

3. Message Passing

Since memory is distributed, processes communicate by sending and receiving messages. Data is explicitly transferred from one process to another through the network.

4. Message Passing Library

MPI provides a standard message passing library that offers functions for commutation and synchronization. This library hides low-level network details and ensures portability and efficiency.

5. Send/Receive Operations

Processes use send operation to accept incoming messages. These operations enable controlled and synchronized communication between distributed processes.

6. Blocking Communication

In blocking communication, a process waits until the send or receive operation is fully

completed before continuing execution. This ensures synchronization but may reduce parallel efficiency.

7. Non-blocking Communication

In non-blocking communication, a process initiates a send or receive operation and continues executing other tasks while the data transfer occurs in the background. This improves performance and parallelism.

Overall, MPI allows multiple processors with separate memories to communicate by passing messages. Its send/receive operations support both blocking and non-blocking communication, enabling efficient and synchronized parallel processing in distributed systems.

Chapter-4 Applications / Use Cases

Here are some key applications and use cases of Java RMI:

- **Distributed Applications**

In this application, different components of a system run on multiple machines and communicate with each other. E.g., University modules such as admissions, library, and grading run on separate servers. When a student registers for a course, the registration request is sent via RMI to all relevant modules to check course availability, update records, and store grades. This simplifies coordination across servers, although performance may depend on network latency.

- **Client-Server Applications**

In this application, clients request services from a central server. E.g., in a banking system, a customer uses a Java client to check their account balance or transfer money. The requests are sent via RMI to the bank's remote server, which processes them and sends back the response. This allows transparent remote method calls but the server can become a single point of failure.

- **Remote File Access**

In this application, clients can access, read, or modify files stored on a remote server. E.g., a cloud storage system allows a Java client to upload, download, or edit documents on a remote server using RMI. The advantage is centralized file management though transferring large files may cause network delays.

- **Distributed Databases**

In this application, databases are spread across multiple servers, allowing centralized queries and updates. E.g., a retail inventory system lets branch offices query and update stock records on a central database server via RMI. While this allows centralized control, maintaining data consistency can be challenging.

- **Collaborative Applications**

In this application, multiple users work together on shared resources in real-time. E.g., team members edit a shared document stored on a server. Updates from one client are sent via RMI and reflected for all other clients immediately. This allows real-time collaboration but requires efficient synchronization.

- **Middleware Services**

In this application, RMI provides services like naming, registry or messaging in

distributed systems. E.g, RMI Registry allows clients to look up remote objects by name to access services on a server. This simplifies service discovery, though it is limited to Java environments.

- **E-commerce Applications**

In this application, clients interact with a central server for product catalog management, order processing, and payments. E.g, an online shopping platform lets customers query inventory, place orders, and make payments through RMI calls to the server. While this enables smooth client-server operations, scaling to very large user bases may require additional architecture.

- **Remote Monitoring and Management**

In this application, administrators monitor and control machines or devices remotely. E.g, in a factory, sensors send machine status to a central server, and admins send control commands to machines via RMI. This supports remote administration but relies on network reliability.

MPI is widely used in distributed system and high-performance computing environments to enable parallel processing across multiple processors. Some key applications includes :

- **Scientific Simulations**

MPI is used for large-scale simulations in physics, chemistry and biology. E.g, climate change modeling, particle collision and molecular dynamics. Its main advantage is high performance computation by distributing tasks across multiple nodes, which reduces execution time and handles complex calculations efficiently.

- **Weather Forecasting**

MPI enables weather forecasting models to run in parallel on supercomputers. It efficiently processes massive atmospheric and climate datasets. The main advantage is faster weather predictions with more accurate and reliable results.

- **Big Data Analysis**

MPI is used to process large datasets by distributing computations across multiple processors. This parallel approach improves efficiency and reduces processing time. E.g, parallel statistical analysis and genome sequencing, where large volumes of data must be analyzed quickly and accurately.

- **Parallel Numerical Computation**

MPI distributes tasks such as matrix multiplication, solving linear equations and numerical simulations across multiple processors. This significantly reduces computation time and improves performance.

- **Machine Learning and AI**

MPI supports distributed training of machine learning models on multiple nodes. E.g, deep neural networks can be trained efficiently on GPU clusters.

- **Supercomputing Tasks**

MPI is widely used in clusters and supercomputers for engineering simulations, astrophysics and financial modeling. Its main advantage is scalability and efficiency for extremely large computations.

Overall, Java RMI simplifies distributed computing by automatically handling exceptions, object serialization, and parameter passing. Compared to technologies like CORBA or Web Services, RMI is Java-specific but offers a simpler, type-safe, and object-oriented approach to building scalable distributed applications while, MPI is a powerful parallel programming model that allows multiple processors to work together on a single problem. By dividing tasks and enabling fast communication between nodes, MPI improves speed and efficiency.

Chapter-5 Advantage and Limitation

Advantages of Java RMI

- 1. Simplifies Remote Communication:** Developers can call methods on remote objects as if they were local, without handling low-level network details.
- 2. Object-Oriented Approach:** RMI fully supports Java objects, allowing easy integration of remote objects in distributed systems.
- 3. Automatic Serialization:** Objects passed between client and server are automatically serialized and deserialized.
- 4. Exception Handling:** Network and remote errors are handled through RemoteException, providing robust communication.
- 5. Platform Independent:** Works across any machine running the Java Virtual Machine (JVM).
- 6. Integration with Java API:** RMI is part of Java, so it integrates seamlessly with existing Java libraries and applications.
- 7. Supports Distributed Applications:** Enables building client-server systems, collaborative tools, and remote monitoring applications efficiently.

Advantage of MPI

- 1. Efficient Communication:** Enables fast and reliable message passing between processes in distributed memory systems.
- 2. Scalability:** Works efficiently for both small clusters and large supercomputers.
- 3. Portability and Language Independence:** Supports C, C++, Fortran, and Java, and runs on multiple platforms.
- 4. Flexible Communication:** Supports both blocking and non-blocking operations, allowing optimized parallel execution.
- 5. Standardized Library:** Provides a consistent set of functions for communication and synchronization, ensuring portability and reliability.

Limitations of Java RMI

- 1. Java-Specific:** Only works with Java applications; not suitable for heterogeneous environments without additional middleware.

- 2. Performance Overhead:** Serialization, network communication, and remote calls can be slower than local method calls.
- 3. Network Dependency:** Remote calls depend on network reliability; failures can disrupt application functionality.
- 4. Security Concerns:** Exposing remote objects over a network can create security risks if not properly managed.
- 5. Scalability Limitations:** For very large-scale distributed systems, RMI may require additional architecture and load balancing.
- 6. Firewall/Port Issues:** RMI uses dynamic ports for communication, which may be blocked by firewalls.

Limitation of MPI

- 1. Complex Programming:** Developers must manually manage message passing and process synchronization.
- 2. Debugging Challenges:** Concurrency issues make testing and debugging difficult.
- 3. Performance Overhead:** Frequent communication or poor design can slow down execution.
- 4. Network Dependency:** Network latency and bandwidth can limit performance in large-scale systems.
- 5. Heterogeneous Systems:** MPI works best on homogeneous clusters; additional effort is needed for mixed environments.

Chapter-6 Real-Time Case Examples

1. Cloud Storage System

RMI enables remote access to files stored on central servers for uploading, downloading, and editing documents.

Advantage: Centralized file management with real-time access for multiple users.

Limitation: Large file transfers may lead to network delays.

2. Retail Inventory Management

Branches of a retail chain update and query stock levels on a central database via RMI.

Advantage: Real-time inventory updates and consistency across multiple locations.

Limitation: Maintaining data consistency can be challenging.

3. Collaborative Editing Tools

Multiple users can work on shared documents stored on a central server. Updates from one client are instantly reflected on others using RMI.

Advantage: Enables real-time collaboration.

Limitation: Requires efficient synchronization to avoid conflicts.

4. E-commerce Platforms

Clients interact with central servers for product browsing, orders, and payments. RMI enables seamless remote method calls.

Advantage: Smooth front-end to back-end operations.

Limitation: Scaling to a large number of users may need additional architecture.

5. Remote Monitoring and Control in Factories

Sensors send machine status to a central server, and administrators send control commands remotely via RMI.

Advantage: Supports real-time monitoring and control.

Limitation: Dependent on network reliability.

6. Distributed Gaming Servers

RMI maintains synchronized game states for multiple players interacting with a central server.

Advantage: Ensures consistent gameplay and real-time updates.

Limitation: High network latency can affect performance.

Chapter-7 Conclusion

Java RMI simplifies distributed computing by allowing Java objects to invoke methods on remote objects as if they were local, handling networking, serialization, and exceptions automatically. It is widely used in client-server systems, collaborative tools, cloud storage, e-commerce, and real-time monitoring applications. While RMI offers platform independence, object-oriented communication, and seamless integration with Java, it is limited to Java environments and depends on network reliability. Overall, RMI is a robust and efficient solution for building scalable and maintainable distributed applications.

MPI enables efficient communication between multiple processors in distributed memory systems through message passing. It supports blocking and non-blocking operations, is portable across platforms, and is widely used in scientific simulations, big data, and supercomputing. Despite its programming complexity and network dependence, MPI provides scalable, fast, and coordinated parallel processing, making it essential for high-performance distributed computing.

Reference

[1] Online Document on JAVA RMI

“JAVA RMI CASE STUDY 1-2,” Scribd, 2023. [Online]. Available: <https://www.scribd.com/document/741565347/JAVA-RMI-CASE-STUDY-1-2>. [Accessed: Jan. 19, 2026].

[2] Advanced Java (J2EE) Notes

V. Vadivazhagan, Advanced Java (J2EE), Annamalai University, 2024. [Online]. Available: [https://annamalaiuniversity.ac.in/studport/download/sci/cis/resources/Programme_MS ci_Software_Engineering_Five_Year_Integrated_PG_Year_IV_Semester_VIII_Course_Code_ISET82_Course_Name_ADVANCED_JAVA_\(J2EE\)_by_Vadivazhagan_K.pdf](https://annamalaiuniversity.ac.in/studport/download/sci/cis/resources/Programme_MS ci_Software_Engineering_Five_Year_Integrated_PG_Year_IV_Semester_VIII_Course_Code_ISET82_Course_Name_ADVANCED_JAVA_(J2EE)_by_Vadivazhagan_K.pdf). [Accessed: Jan. 19, 2026].

[3] Unit-5 Presentation (Scribd)

Unit-5, Scribd Presentation, 2024. [Online]. Available: <https://www.scribd.com/presentation/808266287/unit-5#page=1>. [Accessed: Jan. 19, 2026].

[4] Medium Article on MPI

A. Kumar, Message Passing Interface (MPI), Medium, Apr. 06, 2021. [Online]. Available: <https://arshitkumar-96339.medium.com/message-passing-interface-mpi-88ca9bb14fd8>. [Accessed: Jan. 19, 2026].

[5] YouTube Video on MPI

Message Passing Interface Tutorial, YouTube Video, 2022. [Online]. Available: <https://www.youtube.com/watch?v=rLG82gQj4Ws>. [Accessed: Jan. 19, 2026].